body { font-family: Arial; font-size: 10pt; color: #1a1a1a; line-height: 1.45; } h1 { font-size: 20pt; color: #1e3a5f; } h2 { font-size: 14pt; color: #2c5282; margin-top: 14px; } h3 { font-size: 11pt; color: #2d3748; } code, pre { font-family: Consola; font-size: 8pt; background-color: #f4f4f4; } table { border-collapse: collapse; width: 100%; font-size: 9pt; } th, td { border: 1px solid #cbd5e0; padding: 4px 6px; } th { background-color: #edf2f7; } hr { color: #cbd5e0; }

Smart Vehicle Gate System - Complete Project Guide

Final Year Project (FYP)
Live website: https://automated-gate-access.vercel.app
Edge device: Orange Pi 5 Pro (local AI)
Last updated: June 2026

Table of Contents

1. [What This Project Is]
2. [Big Picture - How Everything Connects]
3. [Hardware You Use]
4. [Software Parts of the System]
5. [The Three AI Models (Explained Simply)]
6. [How One Image Is Processed (Step by Step)]
7. [Weather Detection & Preprocessing]
8. [Orange Pi Folder - Every File Explained]
9. [Setting Up the Orange Pi (First Time)]
10. [Day-to-Day Commands on the Pi]
11. [Uploading Results to the Website]
12. [The Website - Pages & What They Do]
13. [How Data Travels from Pi to Vercel]
14. [Vehicle Access Rules (Allow / Deny)]
15. [Environment Variables & Secrets]
16. [Deployment (Vercel + Pi Updates)]
17. [Demo Workflows (Photos, Videos, Camera)]
18. [What to Expect - Performance & Limits]
19. [Problems We Fixed During Development]
20. [Troubleshooting Guide]
21. [Quick Command Cheat Sheet]
22. [Glossary]

---

1. What This Project Is

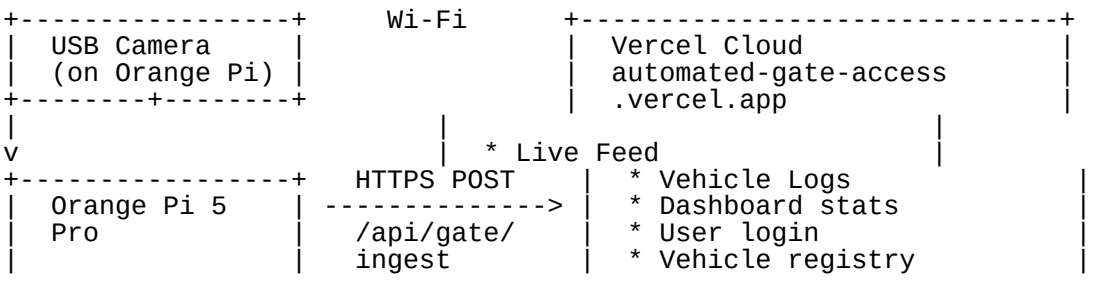
This is an **automated vehicle gate system** for a university Final Year Project. The idea:

1. A **camera** captures a car at a gate.
2. **Artificial intelligence** on an **Orange Pi** (small computer) reads the **license plate** and identifies the **vehicle make/model**.
3. Results are sent to a **web dashboard** on **Vercel** where an admin can see a **Live Feed**, logs, and statistics.
4. Optionally, the system can **allow or deny** access based on registered plates (gate logic + LED).

Important design choice: All three AI models run **on the Orange Pi**, not in the cloud. The cloud only receives the **results** (plate, vehicle, small image). This is called **edge AI** - fast, private, and works even with slow internet for inference.

---

2. Big Picture - How Everything Connects





Two ways to use the system

Mode	Where AI runs	When to use
Pi local (main)	Orange Pi	Real demo, camera, batch photos/videos
Cloud predict (optional)	Hugging Face Space	Manual refresh from website without Pi

Your main workflow uses the **Pi local** path via `gate_sync.py`.

3. Hardware You Use

Device	Role
Orange Pi 5 Pro (16GB)	Runs Python, OpenCV, PyTorch, all 3 models
USB camera (1080p)	Captures cars - connect to Pi USB port for live use
5V 5A USB-C power supply	Required - weak power causes Pi shutdown under load
HDMI monitor + keyboard	Setup and live preview (runwebcam.py)
Wi-Fi	Pi uploads results to Vercel
LED / GPIO (optional)	Physical gate indicator - separate from camera testing
Windows laptop	Record photos/videos, copy files to Pi via scp or USB

Where to plug the camera

- **For real-time on Pi:** USB camera -> **Orange Pi** (not laptop)
- **For recording first:** USB camera -> laptop -> copy files to Pi -> run batch upload

4. Software Parts of the System

On the Orange Pi (`orange_pi_deploy/`)

Part	File(s)	Job
Configuration	config.py	Paths, thresholds, camera settings
Model loading	modelsloader.py	Load YOLO + EfficientNet weights
Weather	preprocessing.py	Detect fog/rain/night; enhance image
Main AI pipeline	pipeline.py	Plate -> OCR -> vehicle in one flow
Website upload	gatesync.py	Run pipeline + POST to Vercel
Local tools	run*.py	Test images, video, webcam, benchmark
Models	models/.pt, .pth	Downloaded AI weights (~207 MB total)

On Vercel (`webfyp/`)

Part	Location	Job
React frontend	src/pages/	Dashboard, Live Feed, logs, settings
Express API	server/	Auth, events, gate ingest, vehicles
Serverless entry	api/index.js	Vercel runs the Node backend

Part	Location	Job
Config	vercel.json	Routes /api/* to backend

Model weights source

- Hugging Face repo: `abdulhadi111222/fyp-vehicle-models`
- Downloaded on Pi by: `python download_models.py`

5. The Three AI Models (Explained Simply)

Model 1 - License Plate Detector (YOLOv8)

Item	Detail
File	models/platebest.pt
Type	YOLOv8 nano (object detection)
Input	Full car image
Output	Rectangle(s) around license plate(s)
Image size	380 pixels (auto-rounded to 384 by YOLO)

What it does: Finds *where* the plate is in the photo.

Model 2 - Character Recognizer (YOLOv8)

Item	Detail
File	models/charbest.pt
Type	YOLOv8 nano, 38 character classes
Input	Cropped plate image
Output	Each letter/digit with position

What it does: Reads *what* is written on the plate. Characters are sorted left-to-right and formatted as `ABC-1234` (Pakistani-style: up to 3 letters + hyphen + 4 digits).

Model 3 - Vehicle Make & Model (EfficientNet-B4)

Item	Detail
File	models/efficientnetb4best.pth + classes.txt
Type	EfficientNet-B4 classifier
Classes	245 vehicle types
Input	224?224 RGB image
Output	Label like Toyotayarisgen 2018 2020 + confidence %

What it does: Identifies *what car* it is from appearance.

Typical timing on Orange Pi CPU

Stage	Time
Plate detection	~80-200 ms
Character OCR	~30-100 ms per plate
Vehicle classifier	~200-800 ms
Total pipeline	~700-1000 ms per image

6. How One Image Is Processed (Step by Step)

This happens inside `pipeline.py` -> function `_run_pipeline_on_pil()`:

```
Step 1  Load image as RGB
v
Step 2  Weather detection (optional)
* If --preprocess none -> skip, label = "none" (Clear)
* If --preprocess auto -> detect fog/rain/night/clear
v
Step 3  Image enhancement (optional)
* fog  -> defog filter
* rain -> derain filter
* night -> brighten filter
* none -> original image unchanged
v
Step 4  Vehicle classification (EfficientNet)
-> make/model label + confidence
v
Step 5  Plate detection (YOLO plate model)
-> bounding box(es) on plate
v
Step 6  For each plate box:
* Crop plate region
* Run character YOLO on crop
* Sort characters left-to-right
* Format as ABC-1234
v
Step 7  Draw annotated image
* Vehicle name at top
* Green box + plate text on each plate
v
Step 8  Return PipelineResult
* plate_texts, vehicle_label, vehicle_conf
* annotated image, timings, weather info
```

---

7. Weather Detection & Preprocessing

Why weather matters

Bad weather (fog, rain, night) makes plates harder to read. The system can:

1. **Detect** weather from the image (no separate weather API for this).
2. **Enhance** the image before running the 3 models.
3. **Show** a label on the website: Clear, Fog, Rain, or Night.

How detection works (simple explanation)

There is **no trained weather AI model**. The code uses **image statistics**:

- **Sky region** (top 35% of image) - haze, rain streaks
- **Center region** (middle 35%) - car body (avoids false "rain" from car edges)
- **Ground region** (bottom 30%) - wet road glare

It scores four classes: **clear**, **fog**, **rain**, **night**, then picks the winner with safety rules (e.g. reject "rain" if only car edges detected, no sky streaks).

Preprocessing modes

Flag	Meaning	Website shows
--preprocess none	No detection, no filter	Clear (best for normal daylight photos)
--preprocess auto	Detect + apply matching filter	Clear / Fog / Rain / Night
--preprocess fog	Force defog filter	Fog
--preprocess rain	Force derain filter	Rain
--preprocess night	Force night enhance	Night

Recommendation for your car photo dataset: use `--preprocess none` to avoid false Fog labels on clear photos.

8. Orange Pi Folder - Every File Explained

Location on Pi: `~/orange_pi_deploy/`

File	What it does
config.py	All settings: model paths, confidence thresholds, camera resolution
modelsloader.py	Loads the 3 models into memory; classifyvehicle() function
preprocessing.py	Weather detect + fog/rain/night filters + plate text formatting
pipeline.py	Full AI pipeline; returns PipelineResult
gatesync.py	Main upload script - pipeline + send to Vercel Live Feed
downloadmodels.py	Download weights from Hugging Face to models/
testmodels.py	Quick test: do all models load?
runimages.py	Process folder of photos -> save annotated JPGs + CSV locally
runvideo.py	Process video file -> sample frames, save results locally
runwebcam.py	Live camera preview on Pi monitor - no website upload
runbenchmark.py	Measure speed (ms per model) -> JSON report
setuporangepi.sh	One-time install: Python, venv, PyTorch, packages
recordvideo.sh	Record N seconds from USB camera via ffmpeg
requirements.txt	Python package list
ORANGEPISETUP.md	Short setup guide
models/	AI weight files (not in zip - download separately)
recordings/	Your photos and videos
outputs/	Saved results from local runs
venv/	Python virtual environment

9. Setting Up the Orange Pi (First Time)

Step 1 - Copy project to Pi

Copy `orange_pi_deploy.zip` to Pi, unzip:

```
unzip orange_pi_deploy.zip -d ~/
cd ~/orange_pi_deploy
```

Or from Windows PowerShell:

```
scp -r "D:\fyp-complete\FYP COMPLETE\orange_pi_deploy"
hadi@10.107.13.131:~/orange_pi_deploy
```

Step 2 - Run setup script

```
chmod +x setup_orangepi.sh
bash setup_orangepi.sh
```

Takes **15-30 minutes** (downloads PyTorch for ARM).

Step 3 - Activate environment (every new terminal)

```
cd ~/orange_pi_deploy
source venv/bin/activate
```

You must see `(venv)` at the start of your prompt.

Step 4 - Download models

```
pip install huggingface_hub
python download_models.py
```

Step 5 - Verify

```
python test_models.py
```

Expected: all 3 models load, sample inference runs.

Step 6 - Fix line endings (if scripts fail with `$'\r'` error)

```
sed -i 's/\r$//' setup_orangepi.sh record_video.sh
```

Step 7 - Correct system time

```
date
sudo apt-get install -y systemd-timesyncd
sudo systemctl enable systemd-timesyncd
sudo systemctl start systemd-timesyncd
```

If timedatectl shows RTC errors, that is OK as long as date shows the correct time.

---

10. Day-to-Day Commands on the Pi

Always start with:

```
cd ~/orange_pi_deploy
source venv/bin/activate
```

Test models only (no camera, no website)

```
python test_models.py
```

Process photos locally (save to `outputs/results/`)

```
python run_images.py recordings/car_photos/ --preprocess none
```

Live camera on Pi monitor (no website)

```
python run_webcam.py --camera 0 --width 1280 --height 720 --preprocess none
```

Press **Q** or **ESC** to quit.

Record a short video from camera

```
bash record_video.sh 15 0
```

---

11. Uploading Results to the Website

Set connection (each terminal session, or add to `~/.bashrc`)

```
export GATE_API_URL="https://automated-gate-access.vercel.app/api"
export GATE_DEVICE_TOKEN="fyp_gate_secret-2026"
```

The token must **exactly match** GATE_DEVICE_TOKEN in Vercel project settings.

Upload all photos in a folder

```
python gate_sync.py --folder recordings/car_photos/ --preprocess none --delay 5
```

- One **Live Feed event** per image
- --delay 5 = 5 second pause between uploads (avoids timeouts)
- --preprocess none = Clear weather on website

Upload one photo

```
python gate_sync.py --image recordings/car_photos/photo.jpg --preprocess none
```

Upload from video (sampled frames)

```
python gate_sync.py --video recordings/car_photos/myvideo.mp4 --every 1 --max-frames 5
--preprocess none --delay 5
```

- --every 1 = one frame every 1 second
- --max-frames 5 = stop after 5 frames (good for one car per video)

All videos in folder (loop)

```
for f in recordings/car_photos/*.mp4; do
python gate_sync.py --video "$f" --every 1 --max-frames 5 --preprocess none --delay 5
done
```

Live camera -> website (real-time integration)

```
python gate_sync.py --webcam --camera 0 --interval 20 --preprocess none
```

- Runs pipeline + uploads every **20 seconds**
- Shows preview window on HDMI
- Press **Q** to quit

Test without uploading

```
python gate_sync.py --image photo.jpg --preprocess none --dry-run
```

Clear all logs before fresh upload

From website: Live Feed -> **Clear All** (admin login)

From Pi:

```
curl -s -X POST "https://automated-gate-access.vercel.app/api/gate/clear" \
-H "X-Gate-Token: fyp_gate_secret-2026"
```

---

12. The Website - Pages & What They Do

URL: https://automated-gate-access.vercel.app

Public pages

Page	Purpose
Home	Landing / introduction
Login	Sign in to dashboard
Signup	Create account (first user becomes admin)

Dashboard pages (after login)

Page	Who sees it	What it shows
Dashboard	Admin	Today's vehicle counts, authorized/unauthorized, gate status, weather widget
Live Feed	Everyone	Real-time events from Pi - plate, vehicle, image, weather, allow/deny
Vehicle Logs	Admin	Full history with search and filters
Manual Override	Admin	Register allowed/denied plates; manage access list
Add Temporary Visitor	Regular users	Add time-limited visitor access
Weather	Everyone	Live weather widget for gate location
Settings	Admin	Notification preferences
Admin Access	Admin	Grant admin privileges to other users

Live Feed features

- Auto-refresh toggle
- Filter by status (allowed / denied / temporary)
- **Clear All** button - removes all events (keeps registered vehicles)
- Each card shows: plate, make/model, confidence, weather, timestamp, snapshot

13. How Data Travels from Pi to Vercel

Step-by-step

Pi runs `gate_sync.py` -> calls `pipeline.py` on an image/frame.

Results packaged into JSON: `json { "plate_texts": ["ABC-1234"], "vehicle_label": "Toyota_yaris_gen 2018 2020", "vehicle_conf": 0.56, "preprocess_mode": "none", "image_base64": "...", "timestamp": "2026-06-20T12:00:00+00:00", "source": "orange_pi_local" }`

Image compressed before upload (~100 KB JPEG) to avoid HTTP 413 errors on Vercel.

POST request sent to: `https://automated-gate-access.vercel.app/api/gate/ingest Header: X-Gate-Token: fyp_gate_secret-2026`

Vercel backend (`gateController.js`): - Checks token - Normalizes plate text - Checks if plate is registered (allow/deny/temp) - Saves event to in-memory storage - Returns `{ event, gate_action: "opened" | "closed" }`

Website reads events via `GET /api/events` and displays on Live Feed and Dashboard.

What is NOT sent to the cloud

- Raw full-resolution video stream
- Model weights
- Unprocessed bulk data

Only: plate text, vehicle label, confidence, weather label, small compressed snapshot, timestamp.

14. Vehicle Access Rules (Allow / Deny)

When an event arrives at Vercel, `resolveAccessStatus()` decides:

Situation	Result	Gate action
Plate in registry with Allow	Allowed	opened

Situation	Result	Gate action
Plate in registry with Deny	Denied	closed
Temporary visitor within valid dates	Allowed	opened
Temporary visitor expired	Denied	closed
Unknown plate, vehicle confidence ? 85%	Allowed	opened
Unknown plate, confidence < 85%	Denied	closed

Register plates under **Manual Override** on the website before demo.

Note: For camera-only testing on Pi, you can ignore gate rules and use `run_webcam.py` locally without uploading.

15. Environment Variables & Secrets

On Orange Pi (terminal export or `~/.bashrc`)

Variable	Example	Purpose
GATEAPIURL	https://automated-gate-access.vercel.app/api	Where to POST results
GATEDEVICETOKEN	fypgatesecret-2026	Shared secret with Vercel
HFTOKEN	(optional)	If Hugging Face repo is private

On Vercel (Project -> Settings -> Environment Variables)

Variable	Purpose
GATEDEVICETOKEN	Must match Pi token
JWTSECRET	Signs login tokens
ADMINEMAILS	Emails auto-promoted to admin
HFSPACEID	Hugging Face Space for optional cloud predict
HFTOKEN	Hugging Face API token

After changing Vercel env vars -> **Redeploy** the project.

16. Deployment (Vercel + Pi Updates)

Vercel (website)

- Code lives in GitHub repo: `AbdullahYaseen01/webfyp`
- Push to `main` -> Vercel auto-deploys
- Frontend (React) + backend (Express) deploy together via `vercel.json`

Update Pi scripts from Windows

Single file:

```
scp "D:\fyp-complete\FYP COMPLETE\orange_pi_deploy\gate_sync.py"
hadi@10.107.13.131:~/orange_pi_deploy/gate_sync.py
```

All three main scripts (recommended - always update together):

```
scp "D:\fyp-complete\FYP COMPLETE\orange_pi_deploy_update.zip"
hadi@10.107.13.131:~/Downloads/
```

On Pi:

```
unzip -o ~/Downloads/orange_pi_deploy_update.zip -d ~/orange_pi_deploy/
python3 -m py_compile ~/orange_pi_deploy/gate_sync.py ~/orange_pi_deploy/pipeline.py
```

```
~/orange_pi_deploy/preprocessing.py
```

Important: Never paste large patch scripts directly on the Pi terminal - they get truncated and break Python files. Always use scp from Windows.

---

17. Demo Workflows (Photos, Videos, Camera)

Workflow A - Record on laptop, upload from Pi

1. Record photos/videos with Windows Camera app
2. Copy to Pi: recordings/car_photos/
3. Clear old logs (optional)
4. Run folder upload + video loop (see Section 11)
5. Open Live Feed on website

Workflow B - Camera on Pi, local test only

1. Plug USB camera into Pi
2. python run_webcam.py --camera 0 --preprocess none
3. Point at car/plate - see detection on HDMI monitor

Workflow C - Camera on Pi, live to website

1. Set env vars
2. python gate_sync.py --webcam --camera 0 --interval 20 --preprocess none
3. Open Live Feed with auto-refresh ON

Workflow D - FYP presentation day

1. Clear Live Feed
2. Register 2-3 demo plates (one allow, one deny)
3. Start webcam sync OR show pre-uploaded batch
4. Show: Pi terminal (inference time) + website (Live Feed event appears)

---

18. What to Expect - Performance & Limits

Expectation	Reality
Real-time 30 FPS video on website	No - system sends one event every N seconds
Inference speed on Pi	~1 image per second (CPU)
Upload interval (webcam)	Default 20 seconds (configurable)
Image upload size	Compressed to ~100 KB (JSON POST)
Models in cloud	No - all on Pi
Works offline for AI	Yes - needs internet only for upload
Vercel stores events forever	In-memory + JSON file; clears on redeploy/cold start in some cases

YOLO warning you may see (normal)

```
WARNING imgsz=[380] must be multiple of max stride 32, updating to [384]
```

This is harmless - YOLO auto-adjusts image size.

---

19. Problems We Fixed During Development

Problem	Cause	Fix
HTTP 413 on upload	Full-size base64 image too large for Vercel	Compress image in gatesync.py (~100 KB)
401 Unauthorized	Pi token ? Vercel GATEDEVICETOKEN	Match both exactly; redeploy Vercel
AttributeError weatherconfidence	Updated gatesync.py but old pipeline.py on Pi	Update all 3 files together via zip
NameError sat in preprocessing	Bug in decideweather()	Fixed missing variables
False Fog on clear photos	Weather always detected even with --preprocess none	Skip detection when mode is none
Dashboard shows 0	Browser "today" filter vs UTC Pi timestamps	Server stats endpoint with UTC + fallback
Batch stops on one error	No try/except on pipeline	Continue on failure, log error
False Rain on car photos	Vertical car edges mistaken for rain	Regional weather classifier (sky/center/ground)
Pi RTC error	Hardware clock uninitialized	Use NTP; date correct is enough
gatesync.py broken on Pi	Truncated paste patches	Restore via scp or orangepideployupdate.zip
Videos ignored by --folder	Folder mode only reads images	Use --video separately for .mp4

20. Troubleshooting Guide

Symptom	Solution
No module named torch	source venv/bin/activate
401 on upload	Check GATEDEVICETOKEN on Pi and Vercel
413 on upload	Update gatesync.py (compression fix)
Cannot open camera 0	v4l2-ctl --list-devices, try --camera 1
No OpenCV window	pip install opencv-python (not headless)
Pi shuts down under load	Use 5V 5A PSU; close other apps
$'\r': command not found	sed -i 's/\r$//' script.sh
apt "not valid yet"	Fix clock: sudo timedatectl set-ntp true
Model files 0 bytes	Re-run python downloadmodels.py
Dashboard empty, logs full	Hard refresh; wait for Vercel redeploy of stats fix
Duplicate events on Live Feed	Use Clear All; increase --delay; dedup is 10-second window
Everything shows Fog/Rain	Use --preprocess none for daylight

21. Quick Command Cheat Sheet

```
# --- Every session ---
cd ~/orange_pi_deploy && source venv/bin/activate
export GATE_API_URL="https://automated-gate-access.vercel.app/api"
export GATE_DEVICE_TOKEN="fyp_gate_secret-2026"

# --- Local test ---
python test_models.py
python run_webcam.py --camera 0 --preprocess none
python run_images.py recordings/car_photos/ --preprocess none

# --- Upload to website ---
python gate_sync.py --folder recordings/car_photos/ --preprocess none --delay 5
python gate_sync.py --video recordings/video.mp4 --every 1 --max-frames 5 --preprocess
none --delay 5
python gate_sync.py --webcam --camera 0 --interval 20 --preprocess none

# --- Clear website logs ---
curl -s -X POST "$GATE_API_URL/gate/clear" -H "X-Gate-Token: $GATE_DEVICE_TOKEN"
```

```
# --- Check camera ---
v4l2-ctl --list-devices
ls /dev/video*
```

22. Glossary

Term	Meaning
Edge AI	AI runs on local device (Pi), not in cloud
VMMR	Vehicle Make and Model Recognition
LPD	License Plate Detection
OCR	Optical Character Recognition (reading plate characters)
YOLO	You Only Look Once - fast object detection
EfficientNet	Deep learning image classifier
Vercel	Cloud hosting for the website + API
Live Feed	Real-time list of detected vehicles on website
gatesync.py	Script that runs AI on Pi and uploads to website
preprocess none	Skip weather detection; show Clear on website
Ingest	API endpoint that receives Pi detection results
GATEDEVICETOKEN	Secret password between Pi and Vercel

Project File Map (Quick Reference)

```
FYP COMPLETE/
+-- orange_pi_deploy/          <- Everything that runs on Orange Pi
|   +-- gate_sync.py           <- Pi -> Vercel upload (main)
|   +-- pipeline.py            <- 3-model AI pipeline
|   +-- preprocessing.py       <- Weather + image filters
|   +-- models_loader.py       <- Load AI weights
|   +-- config.py              <- Settings
|   +-- run_webcam.py          <- Live camera (local only)
|   +-- run_images.py          <- Batch photos (local)
|   +-- run_video.py           <- Video frames (local)
|   +-- download_models.py     <- Get weights from Hugging Face
|   +-- models/                <- plate_best.pt, char_best.pt, efficientnet...
|   +-- recordings/            <- Your photos and videos
|
+-- webfyp/                    <- Website (GitHub -> Vercel)
+-- src/pages/             <- React UI pages
+-- server/                <- Express API backend
|   +-- controllers/gateController.js  <- Pi ingest + access rules
|   +-- utils/storage.js   <- Event storage
+-- vercel.json            <- Deployment config
```

End of document.

For questions during demo prep, refer to the section numbers above or the Quick Command Cheat Sheet (Section 21).